Benha University

Benha Faculty Of Engineering

Electrical Engineering Department

PROJECT

HUFFMAN CODE IMPLEMENTATION

PREPARED BY

MOHAMMAD PRINCE ABDELRAHMAN ELSAYED

MAZEN NASSER AHMED

MOHAMMAD ELSAYED GAMAL

SUPERVISOR

DR. NADA ELMELIGY

TABLE OF CONTENTS

INTRODUCTION

In the modern era of digital communication, the demand for efficient data storage and transmission has become paramount. **Huffman Coding**, developed by David Huffman in 1952, remains one of the most fundamental and widely used algorithms for **lossless data compression**. It serves as a cornerstone in the field of **Information Theory** and is integrated into many contemporary technologies, including JPEG images and MP3 audio files.

The core principle of Huffman Coding lies in **Variable-Length Encoding**. Unlike standard encoding schemes that assign a fixed number of bits to every character, Huffman Coding assigns shorter binary codes to symbols that appear more frequently and longer codes to those that occur rarely. This process is driven by the **source statistics**, where a unique prefix-free tree (Huffman Tree) is constructed to ensure that no code is a prefix of another, allowing for unambiguous decoding.

By aligning the code lengths with the probability of symbol occurrence, the algorithm minimizes the average code length, aiming to approach the theoretical limit known as **Entropy**.

Objectives of this project:

- To explore the mathematical foundation and logic behind the Huffman algorithm.
- To demonstrate the step-by-step construction of the Huffman Tree.

METHODOLOGY

The Huffman Coding algorithm is based on the principle of **Optimal Prefix Coding**. The methodology follows a mathematical logic to minimize the weighted path length of a binary tree, ensuring maximum compression efficiency.

1. Information Analysis

The fundamental logic starts with analyzing the input source. Each symbol (character) is assigned a weight based on its **probability of occurrence**. According to Shannon's Information Theory, symbols with higher probabilities contain less "information" and should therefore be represented with fewer bits to optimize space.

2. The Optimal Tree Construction (Greedy Approach)

The core of the methodology is the construction of a **Huffman Tree**, which is a type of Extended Binary Tree. The logic follows these mathematical rules:

- **Bottom-Up Synthesis:** Unlike many algorithms that start from the top, Huffman starts from the least frequent symbols (the leaves) and works toward the root.
- **Sibling Pairing:** At each step, the two symbols with the smallest weights are paired as siblings under a common parent node.
- **Weight Summation:** The parent node inherits a weight equal to the sum of its children's weights, representing the combined probability of that branch.
- **Recursive Reduction:** This process reduces the number of available nodes by one in each iteration until a single "Root" node with a weight of 1.0 (or 100%) is reached.

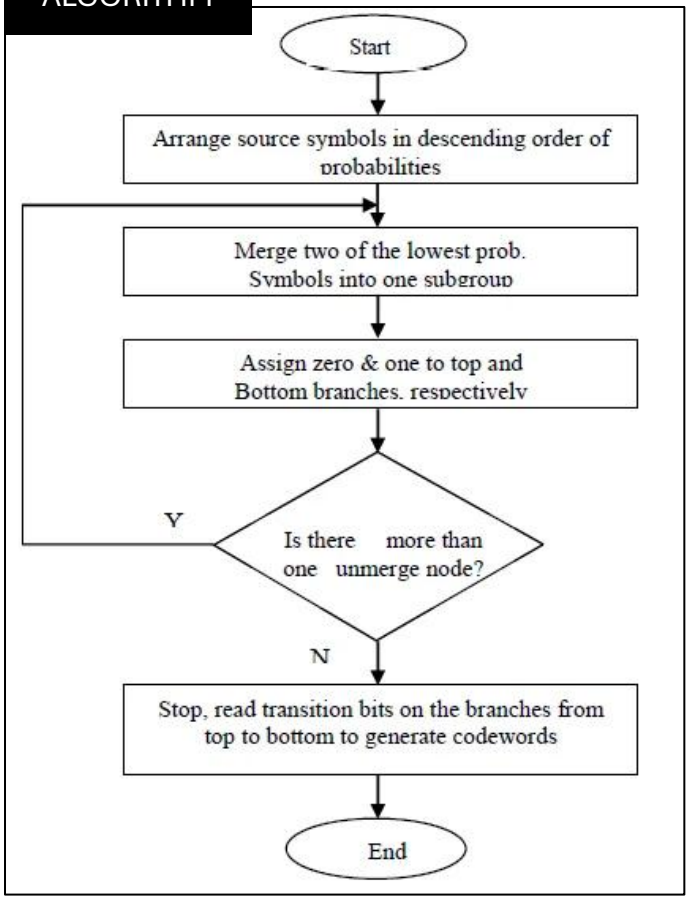
3. Prefix-Free Encoding Logic

The methodology ensures that the resulting codes are **Prefix-Free**. This means no code is a prefix of another (e.g., if 'A' is 01, no other character can start with 01). This is achieved by:

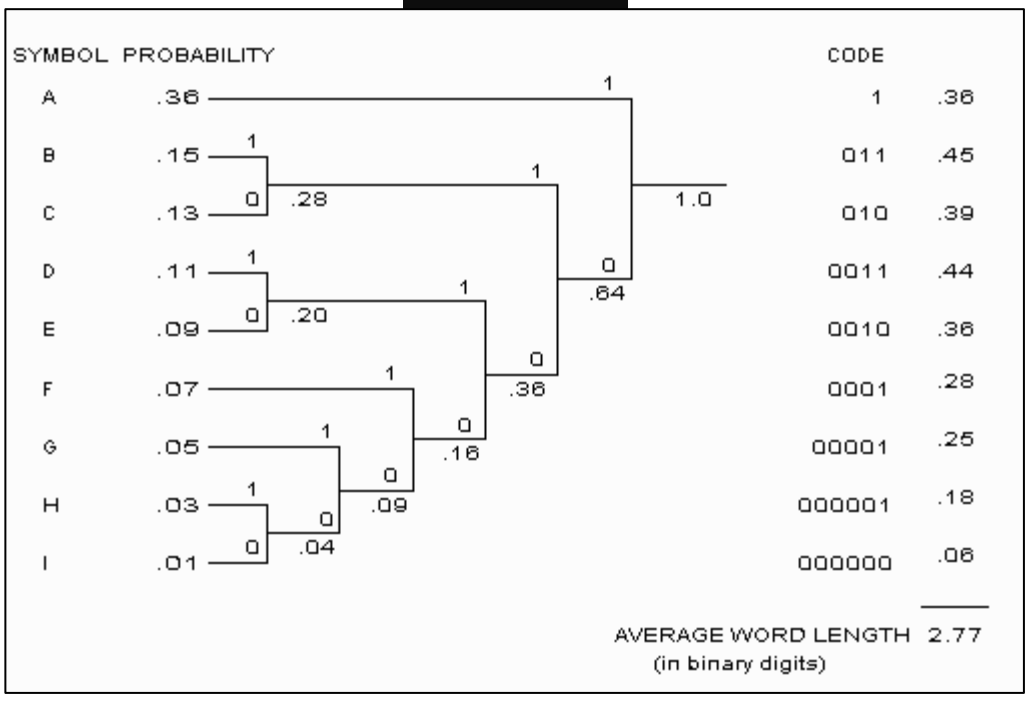
- Assigning a unique binary path to each leaf.
- Standardizing the branching (typically **Left = 0** and **Right = 1**).

This mathematical structure guarantees that the data can be uniquely decoded in a single pass without ambiguity.

ALGORITHM

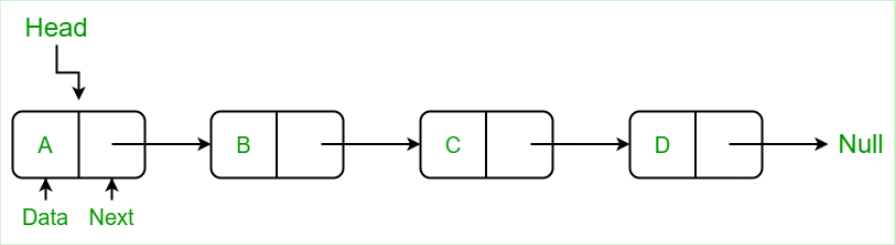


EXAMPLE

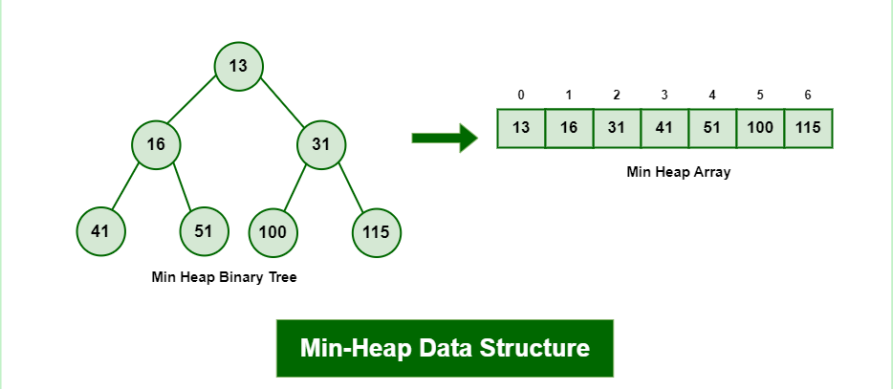


ESSENTIAL PREREQUISITES

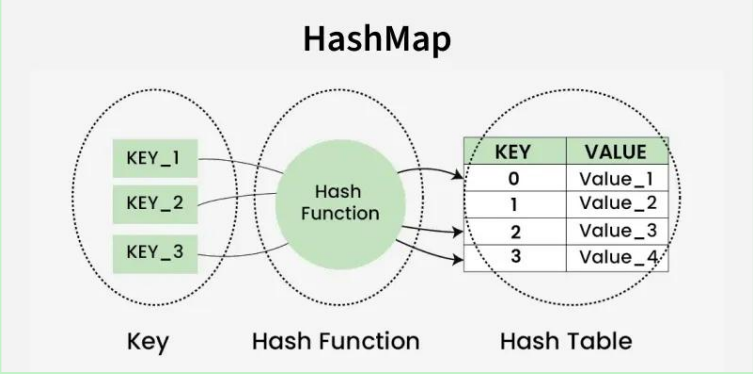
1. **Custom Huffman Node:** A manually defined structure containing the data, frequency, and pointers (*left, *right) to build the tree hierarchy.
2. **Manual Linked List:** Used to manage the connections between nodes and facilitate the recursive structure of the Huffman Tree.



3. **Custom Min-Heap:** A priority-based structure implemented manually to always retrieve the two smallest nodes. It handles the "Up-Heap" and "Down-Heap" operations to maintain order after every insertion and extraction.



4. **Manual Hash Map (Lookup Table):** A custom-built table used to store character frequencies and record the final binary codes, ensuring efficient data retrieval without relying on external libraries.



SOURCE CODE IMPLEMENTATION

The following implementation demonstrates the construction of a Huffman compression engine using **low-level data structures**. By avoiding high-level built-in libraries, this approach focuses on manual memory management and the fundamental logic of tree synthesis, ensuring a deep understanding of the algorithm's internal mechanics.

HUFFMAN NODE

```
#ifndef HUFFMANNODE_H
#define HUFFMANNODE_H

#include <iostream>
using namespace std;

class HuffmanNode {
    public :
        char character;
        int frequency;
        HuffmanNode *left;
        HuffmanNode *right;
        HuffmanNode(char c, int f) {
            character = c;
            frequency = f;
            left = right = NULL;
        }
};

#endif
```

LINKED – LIST NODE

```
#ifndef NODE
#define NODE
#include <iostream>
using namespace std;
template <typename TYPE>
class Node
{
    public :
        char Key;
        TYPE Value;
        Node* next;
        Node(char key,TYPE value)
        {
            Key = key;
            Value = value;
            next = NULL;
        }

};
#endif
```

SINGLE LINKED LIST

A **Single Linked List** is a linear data structure where elements, called **Nodes**, are not stored in contiguous memory locations. Instead, each node consists of Data , next pointer .

```
#ifndef SINGLELL
#define SINGLELL

#include <iostream>
#include "Node.h"
using namespace std;
template <typename TYPE>

class SingleLinkedList{
    private :
        Node<TYPE>* head ;

    public :
        SingleLinkedList(){
            head = NULL;
        }
        bool isEmpty(){
            return (head==NULL);
        }
        void InsertAtFirst(char key,TYPE value){
            Node<TYPE>* newNode = new Node(key,value);
            newNode->next = head;
            newNode->Value = value;
            head = newNode;

        }
        void InsertAtLast(char key,TYPE value){
            if(isEmpty()){
                InsertAtFirst(key , value);
            }else{
                Node<TYPE>* newNode = new Node(key,value);
                Node<TYPE>* temp = head ;
                while(temp->next != NULL){
                    temp = temp->next;
                }
                temp -> next = newNode;

            }
        }

        void DeleteFirstNode(){
            if(!isEmpty()){
                Node<TYPE>* dltptr = head;
                head = head ->next ;
                delete dltptr;
            }else{
                cout<<"Empty List !!\n";
            }

        }
```

```
void DeleteLastNode(){
    if(!isEmpty()){
        if(Count()==1){
            DeleteFirstNode();
        }else{
            Node<TYPE>* dltptr = NULL;
            Node<TYPE>* temp = head;
            while(temp -> next -> next != NULL ){
                temp = temp -> next;
            }
            dltptr = temp -> next;
            temp -> next = NULL;
            delete dltptr;
        }
    }else{
        cout<<"Empty List !!\n";
    }
}
void DeleteAnyNode(char element){
   Node<TYPE>* dltptr = Search(element);
   if(!isEmpty()){
       if(dltptr == head){
           DeleteFirstNode();
       }else if(dltptr->next == NULL){
           DeleteLastNode();
       }else{

           Node<TYPE>* temp = head;
           while(temp->next != dltptr){
               temp = temp -> next;
           }
           temp -> next = dltptr -> next;
           delete dltptr;
       }
   }else{
       cout<<"Empty List !!\n";
   }
}
void Show(){
    if(isEmpty()){
        cout<<"Empty List !!\n";
    }else{
        Node<TYPE>* temp = head;
        while(temp  != NULL){
            cout<<"Key -> "<<temp->Key<<" ===> Value -> "<<temp->Value<<endl;
            temp = temp -> next;
        }
    }
}
int Count(){
    int count = 0;
    if(!isEmpty()){
        Node<TYPE>* temp = head;

        while(temp  != NULL){
            count++;
            temp = temp -> next;
        }
    }
    return count;
}
```

```
        Node<TYPE>* Search(char key){

            if(!isEmpty()){
                Node<TYPE>* temp = head;

                while(temp  != NULL){
                    if (temp->Key == key){
                        return temp;
                    }
                    temp = temp -> next;
                }
            }
            return NULL;
        }

};

#endif
```

MIN-HEAP

The **Min-Heap** is a specialized tree-based data structure that satisfies the **Heap Property**: the value of each parent node is less than or equal to the values of its children. In this project, it serves as the ultimate **Priority Queue**.

```
#ifndef HEAP
#define HEAP
#include <iostream>
#include "HuffmanNode.h"
using namespace std;
#define SIZE 256

class MinHeap
{
private:
    HuffmanNode *Data[SIZE];
    int index;

public:
    MinHeap()
    {
        index = 0;
    }
    int getSize()
    {
        return index;
    }
    bool isEmpty()
    {
        return (index == 0);
    }
    void Insert(HuffmanNode *node)
    {
        Data[index] = node;
        index++;

        int i = index - 1;
        while (i > 0)
        {
            int parent = (i - 1) / 2;
            if (Data[i]->frequency < Data[parent]->frequency)
            {
                swap(Data[i], Data[parent]);
                i = parent;
            }
            else
                break;
        }
    }
```

```
    HuffmanNode *ExtractMin()
    {
        if (isEmpty())
            return NULL;

        HuffmanNode *Min = Data[0];

        Data[0] = Data[index - 1];
        index--;

        int i = 0;
        while (true)
        {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int smallest = i;

            if (left < index && Data[left]->frequency < Data[smallest]->frequency)
                smallest = left;

            if (right < index && Data[right]->frequency < Data[smallest]->frequency)
                smallest = right;

            if (smallest != i)
            {
                swap(Data[i], Data[smallest]);
                i = smallest;
            }
            else
                break;
        }

        return Min;
    }
    void Display()
    {
        for (int i = 0; i < index; i++)
        {
            cout << Data[i]->character << " -> " << Data[i]->frequency << "\n";
        }
    }
};

#endif
```

HASH TABLE

A **Hash Table** is a data structure that maps **keys** to **values** using a special mathematical function called a **Hash Function**. It is designed for ultra-fast searching, insertion, and deletion operations.

```
#ifndef HASH
#define HASH
#include <iostream>
#include "SingleLinkedList.h"
using namespace std;
#define SIZE 256
template <typename TYPE>

class HashMap
{



    public :
        SingleLinkedList<TYPE> HashTable[SIZE];

        HashMap()
        {
            ExistingElements = 0;
        }
        int Hash(char key)
        {
            return ((unsigned char)key%SIZE);
        }
        void Insert(char key , TYPE value)
        {
            int index = Hash(key);
            cout<<index<<endl;
            SingleLinkedList<TYPE>& LL = HashTable[index];
            Node<TYPE>* temp = LL.Search(key);
            if(temp!=NULL)
            {
                temp -> Value = value;
            }else
            {
                LL.InsertAtLast(key,value);

            }
        }
```

```
        TYPE Get(char key)
        {
            int index = Hash(key);
            SingleLinkedList<TYPE>& LL = HashTable[index];
            Node<TYPE>* temp = LL.Search(key);
            if(temp!=NULL)
            {
                return temp->Value;;
            }else
            {
                return TYPE{};
            }
        }
        void Remove(char key)
        {
            int index = Hash(key);
            SingleLinkedList<TYPE>& LL = HashTable[index];
            Node<TYPE>* temp = LL.Search(key);
            if(temp==NULL)
                return ;
            else
            {
                LL.DeleteAnyNode(key);
            }
        }
        void Display()
        {
            cout<<"-=-=-=-=-=-= Table =-=-=-=-=-=-=-"<<endl;
            Node<TYPE>* temp;
            for(int i =0 ;i<SIZE;i++)

           {

                if(!HashTable[i].isEmpty())
                {

                    HashTable[i].Show();

                }


            }
           cout<<"-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-"<<endl;
        }

};
#endif
```

HUFFMAN TREE IMPLEMENTATION

The Construction Algorithm :-

1. **Initial State:** Every character and its frequency starts as a standalone leaf node. These nodes are stored in a **Min-Heap** (Priority Queue).
2. **Iterative Merging:** Extract the two nodes with the **lowest frequencies** from the heap.
    - Create a new **internal node** (Parent) with a frequency equal to the sum of the two nodes.
    - Assign the first extracted node as the **left child** and the second as the **right child**.
    - Insert the new parent node back into the Min-Heap.
3. **Termination:** Repeat the process until only **one node** remains in the heap. This node is the **Root** of the Huffman Tree.

Encoding Strategy :-

Once the tree is built, we traverse it from the root:

- Assign **'0'** for every move to the **left child**.
- Assign **'1'** for every move to the **right child**.
- The sequence of bits from the root to a leaf node forms the **Huffman Code** for that character.

```
#include "MinHeap.h"
#include "HashMap.h"
#include "HuffmanNode.h"
using namespace std;


class HuffmanTree
{
private:
    void GenerateCodes(HuffmanNode *node, string code)
    {
        if (node == NULL)
            return;
        if ((node->left == NULL) && (node->right == NULL))
        {
            codeMap.Insert(node->character, code);
            return;
        }
        GenerateCodes(node->left, code + "0");
        GenerateCodes(node->right, code + "1");
    }

public:
    HuffmanNode *root;
    HashMap<int> freqMap;
    MinHeap heap;
    HashMap<string> codeMap;
    void BuildFrequencyMap(string text)
    {
        for (int i = 0; i < text.length(); i++)
        {
            freqMap.Insert(text[i], freqMap.Get(text[i]) + 1);
        }
    }
    void BuildHeap()
    {
        for (int i = 0; i < SIZE; i++)
        {
            char c = (char)i;
            int freq = freqMap.Get(c);
            if (freq > 0)
            {
                HuffmanNode *node = new HuffmanNode(c, freq);
                heap.Insert(node);
            }
        }
    }
```

```
    void BuildTree()
    {
        while (heap.getSize() > 1)
        {
            HuffmanNode *leftNode = heap.ExtractMin();
            HuffmanNode *rightNode = heap.ExtractMin();
            HuffmanNode *parentNode = new HuffmanNode('\0', leftNode->frequency +
            rightNode->frequency);
            parentNode->left = leftNode;
            parentNode->right = rightNode;
            heap.Insert(parentNode);
        }
        root = heap.ExtractMin();
    }
    void GenerateCodes()
    {
        GenerateCodes(root, "");
    }

    string Encode(string text)
    {
        string code = "";
        for (int i = 0; i < text.length(); i++)
        {
            code = code + "  " + codeMap.Get(text[i]);
        }
        return code;
    }
};
```

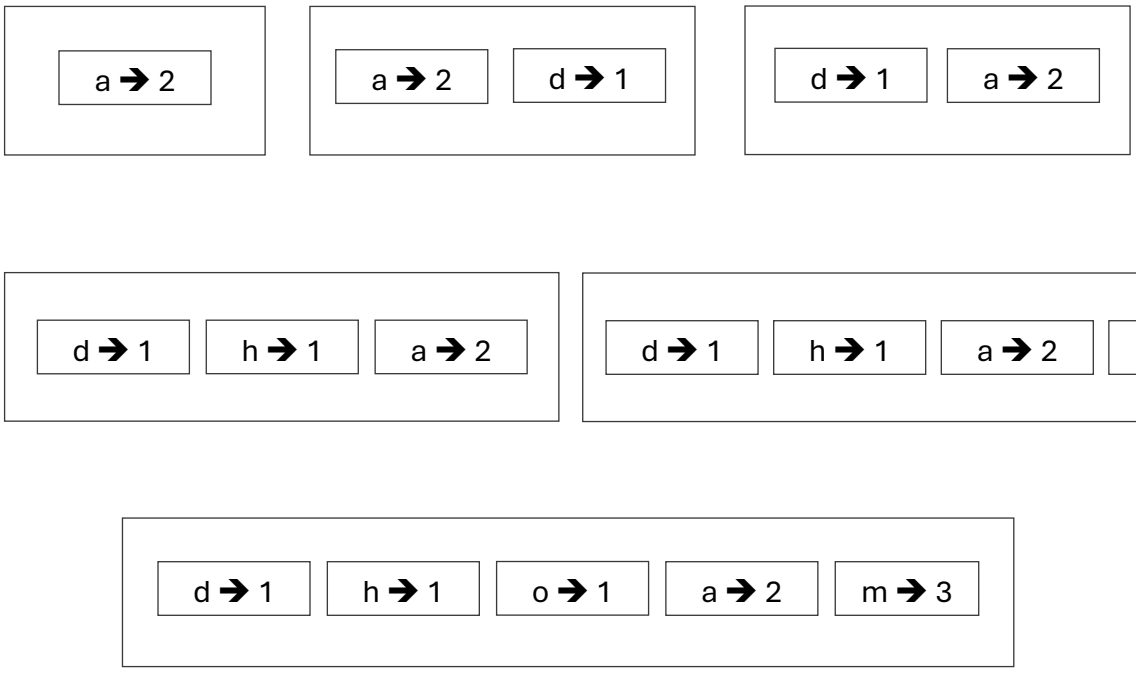
TESTING CODE

Encode “mohammad” Using Huffman Code ? “ 0 1111 110 10 0 0 10 1110 ”

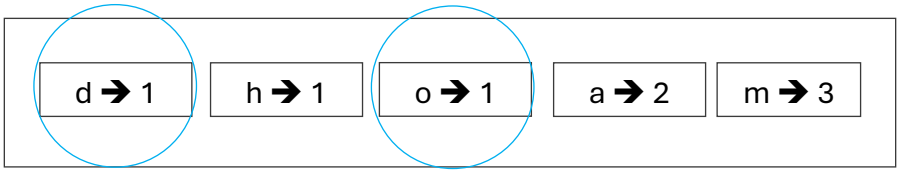
1. Create Table Of Frequencies

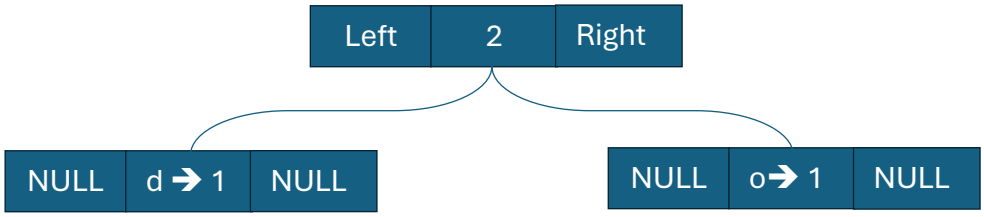
m	3
a	2
h	1
o	1
d	1

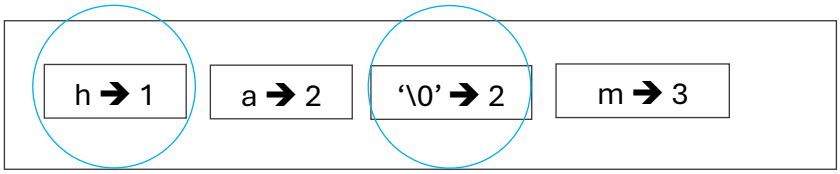
2. Building Heap (Min Heap)

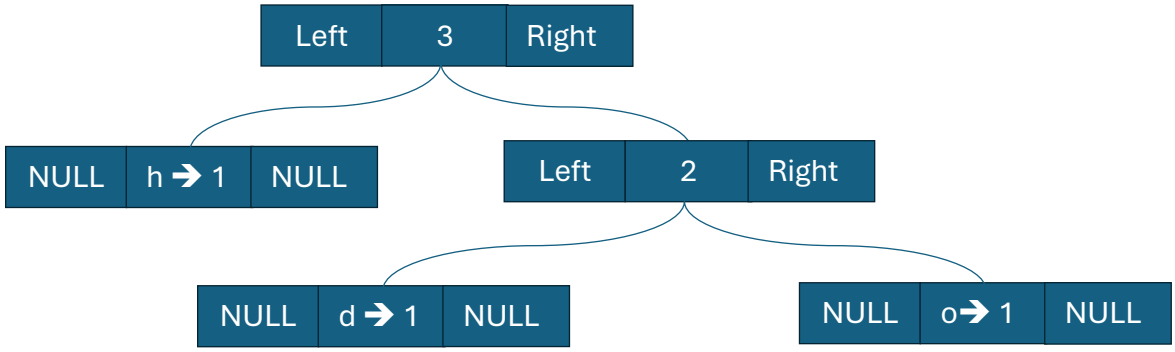


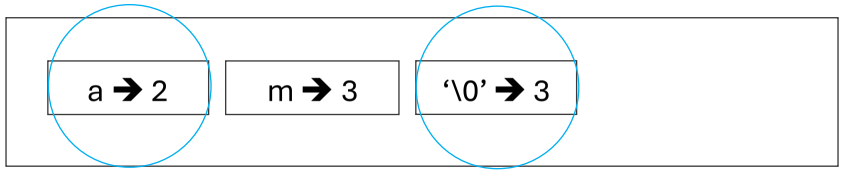
3. Building Tree

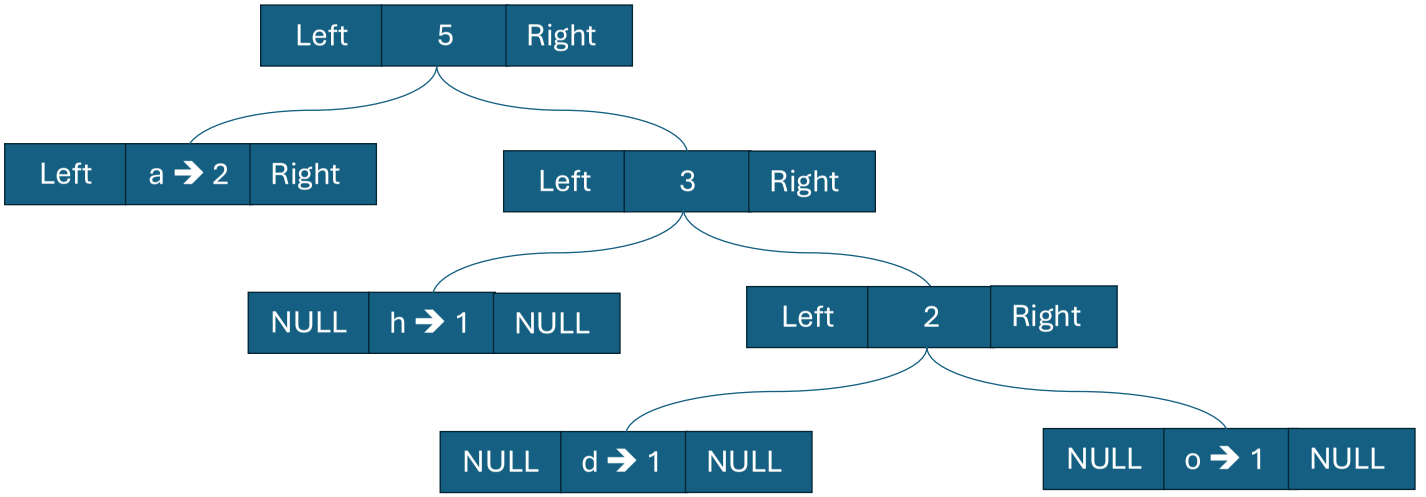


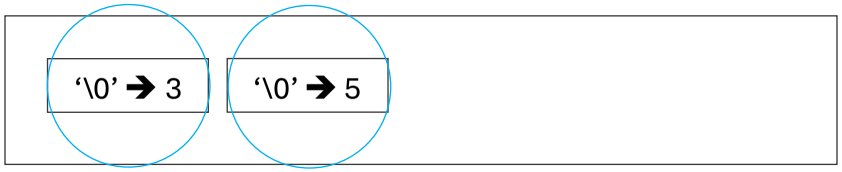


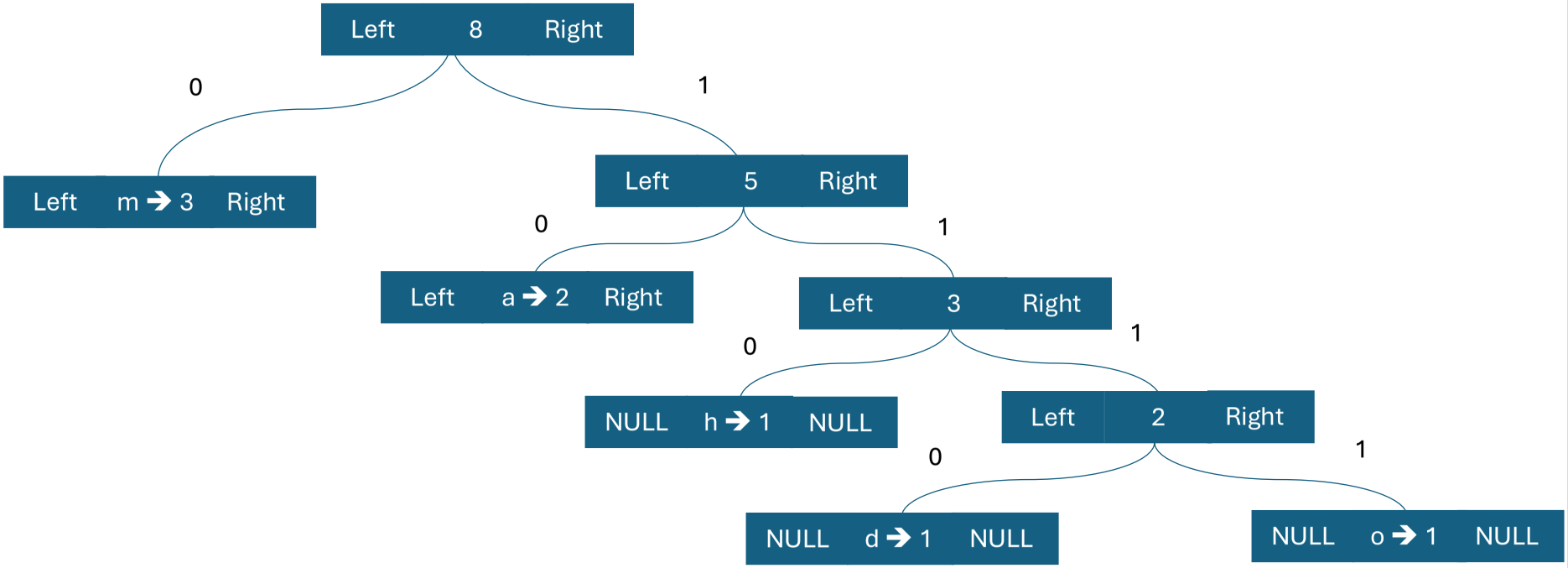





a ➔ 2
m ➔ 3
‘\0’ ➔ 3


Left
5
Right
Left
a ➔ 2
Right
Left
3
Right
NULL
h ➔ 1
NULL
Left
2
Right
NULL
d ➔ 1
NULL
NULL
o ➔ 1
NULL


'\0' ➔ 3
'\0' ➔ 5


Left 8 Right
0
1
Left m ➔ 3 Right
Left 5 Right
0
1
Left a ➔ 2 Right
Left 3 Right
0
1
NULL h ➔ 1 NULL
Left 2 Right
0
1
NULL d ➔ 1 NULL
NULL o ➔ 1 NULL